

FREE RESOURCE · WEB DEVELOPMENT

Web Accessibility Quick Reference

A quick reference guide to WCAG 2.1 accessibility standards — the 12 most important checks for web developers and content creators, with testing tools and common failure patterns.

WCAG 2.1 Overview

12 Key Checks

Testing Tools Guide

Common Failure Patterns

12

key checks

4

POUR principles

5

tool categories

7

failure patterns

WHY THIS MATTERS

Accessibility isn't a checkbox exercise for a niche audience — an estimated 1 in 6 people has some form of disability, and accessible design consistently improves usability for everyone, not only people who rely on assistive technology. These 12 checks won't make a site fully WCAG compliant, but they catch the issues that cause the most real-world harm.

WCAG 2.1 Overview — The Four Principles

WCAG organises every guideline under four principles, often remembered by the acronym POUR. Every one of the checks in this guide maps back to one of these four.

P

Perceivable

Information and interface components must be presentable to users in ways they can actually perceive — not invisible to any of their senses.

O

Operable

Interface components and navigation must be operable by everyone, regardless of input method — keyboard, mouse, touch, or switch device.

U

Understandable

Information and the operation of the interface must be understandable — predictable, clearly labelled, and free of unexplained errors.

R

Robust

Content must work reliably across a wide range of user agents, including current and future assistive technologies.

12 Key Accessibility Checks

Organised by the POUR principle each one supports, with a quick note on how to actually test it.

Perceivable**3 checks**

- 01** ☐ **All images have meaningful alt text.**
How to test: Purely decorative images should use alt="" rather than being left undescribed.
- 02** ☐ **Colour is never the only way information is conveyed.**
How to test: Error states, required fields, and statuses should also use text, icons, or patterns — not colour alone.
- 03** ☐ **Text has sufficient colour contrast against its background.**
How to test: 4.5:1 for normal text, 3:1 for large text (18pt+, or 14pt+ bold).

Operable**3 checks**

- 04** ☐ **All functionality is available using the keyboard alone.**
How to test: Tab through the entire page with no mouse — every interactive element should be reachable and usable.
- 05** ☐ **Focus order is logical, and focus indicators are clearly visible.**
How to test: The visible focus outline should never be removed via CSS without a clear visible replacement.
- 06** ☐ **Users have enough time to read and use content.**
How to test: Avoid unexpected session timeouts, or provide a clear warning and a way to extend the time.

Understandable**3 checks**

- 07** ☐ **Form fields have clear, programmatically associated labels.**
How to test: Placeholder text is not a substitute for a real <label> element — it disappears the moment the user starts typing.
- 08** ☐ **Error messages clearly identify what went wrong and how to fix it.**
How to test: "Invalid input" tells the user nothing useful; "Enter a valid email, e.g. name@example.com" does.
- 09** ☐ **Navigation and layout are consistent across pages.**
How to test: Repeated components (nav, search, footer) should appear in the same relative place and order throughout.

Robust

3 checks

10 ☐ HTML is valid and uses semantic elements.*How to test: Use <nav>, <button>, and <header> rather than generic <div> elements styled to look the same.*11 ☐ ARIA attributes are used correctly, and only when needed.*How to test: A native <button> is always more robust than a <div> with an ARIA role bolted on.*12 ☐ Content works correctly with at least one screen reader.*How to test: Test with NVDA, JAWS, or VoiceOver — automated scanners alone will miss real usability problems.*

Testing Tools Guide

No single tool catches everything — combine automated scanning with genuine manual testing.

Check Type	Tools	Note
Automated scanning	axe DevTools, WAVE, Lighthouse (Accessibility audit)	Catches roughly 30–40% of issues — a starting point, never the full picture.
Colour contrast	WebAIM Contrast Checker, Chrome DevTools contrast ratio	Check text against every background it actually appears on, not just the obvious cases.
Screen readers	NVDA (Windows, free), VoiceOver (Mac / iOS, built-in), JAWS (Windows, paid)	Nothing substitutes for testing with a real screen reader at least once per major release.
Keyboard testing	No special tool needed	Unplug the mouse and Tab through the entire page — the fastest, cheapest accessibility test available.
Browser extensions	axe DevTools, WAVE Evaluation Tool, Accessibility Insights	Run these during development, not only right before launch.

Common Failure Patterns

The same handful of mistakes show up repeatedly across otherwise well-built sites. Watch for these specifically.

✗ Alt text that literally says “image123.jpg”, or is left empty on images that carry real meaning.

✗ Custom dropdowns and modals built entirely from styled <div> elements that a keyboard simply cannot reach.

-
- ✗ Placeholder text used as the only label, so the field's purpose disappears the moment someone starts typing.
 - ✗ Colour-only error indication — a red border with no text explaining what actually needs to change.
 - ✗ Low-contrast light grey text on a white background that looks elegant but fails contrast checks badly.
 - ✗ Auto-playing carousels or background video with no visible pause control.
 - ✗ Focus outlines removed via CSS (outline: none) with no visible replacement put in their place.
-

WHERE TO GO DEEPER

This guide covers the checks with the highest real-world impact, not the full WCAG 2.1 standard. For a complete reference, the W3C's Web Content Accessibility Guidelines and the WebAIM website both offer detailed, freely available criteria and techniques for every success criterion.